

DVA494

Project: Implementing a UART Protocol in VHDL

Spring 2026

1 Overview

This exercise guides the implementation of a Universal Asynchronous Receiver-Transmitter (UART) protocol on the Digilent Basys 3 FPGA board. The design targets the foundational 8N1 format (8 data bits, no parity, 1 stop bit) using a Finite State Machine (FSM) architecture, emphasizing fully synchronous design practices and reliable asynchronous reception through oversampling and synchronization.

Learning Objectives

By the end of this exercise, you should be able to:

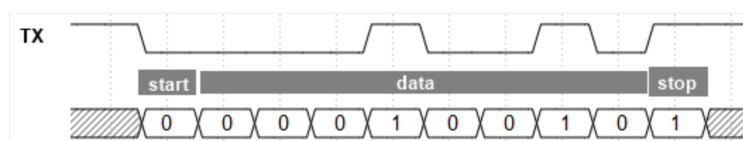
- Explain the 8N1 frame format and its protocol efficiency (approximately 80% payload for a 10-bit frame).
- Compute clock divisors for baud-rate synthesis from a 100 MHz clock.
- Implement a Moore-style transmitter FSM (IDLE, START, DATA, STOP).
- Implement a receiver using $16\times$ oversampling and midpoint sampling, including a 2-FF synchronizer.
- Map UART signals to the correct Basys 3 pins using an XDC constraints file.
- Debug UART bring-up using a terminal emulator and on-board LEDs.

2 Background: The UART Protocol

UART is an asynchronous serial protocol: transmitter and receiver do not share a clock. Timing depends on a pre-established **baud rate** (bit duration). The reliability of the link is contingent on the precision of the oscillators at both ends of the channel

To facilitate data recovery without a shared clock, the UART protocol encapsulates individual bytes into a structured frame. The **8N1 configuration** is structured as follows:

- Idle state: line held high.
- Start bit: low (0) for 1 bit period.
- 8 data bits: typically LSB first, 1 bit period each.
- Stop bit: high (1) for at least 1 bit period.



Baud Rate and Clock Precision

The receiver must reconstruct the timing of the bitstream based solely on its local clock and the detected falling edge of the start bit. For a standard 10-bit frame (1 start + 8 data + 1 stop), baud error on the order of 3–5% is generally the maximum tolerable before framing errors or bit-slips occur.

The precision of the baud rate in an FPGA is determined by the master clock frequency (f_{clk}) and the integer divisor (N) used:

$$\text{Ideal Divisor } N = \frac{f_{clk}}{BR}$$

The Basys 3 board utilizes a 100 MHz master oscillator, which allows common baud rates such as 9600 and 115200 to be synthesized with minimal rounding error. The Basys 3 includes an FTDI FT232HQ USB-to-UART bridge that appears as a Virtual COM Port on the PC, enabling use of standard terminal programs (e.g., PuTTY, Tera Term). This is what we shall use to test the protocol.

UART Transmitter Finite State Machine

The transmitter's role is parallel-to-serial conversion, ensuring each bit's timing adheres strictly to the baud rate. The

FSM utilizes a Moore architecture with four distinct states.

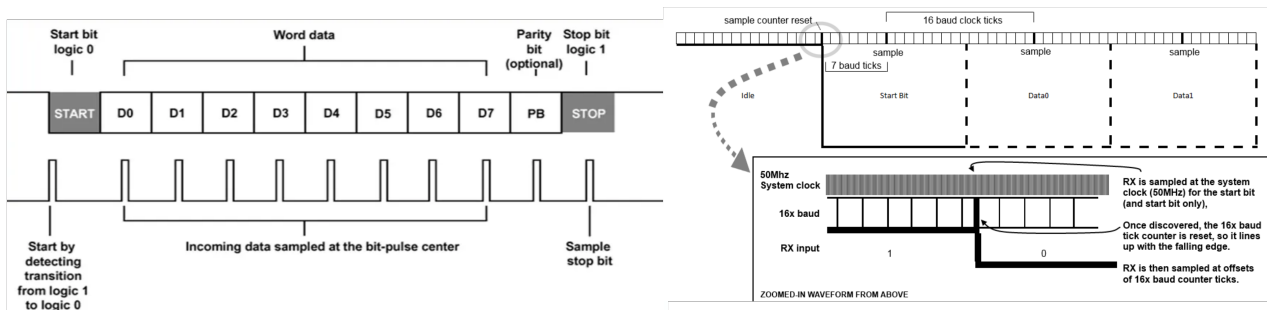
FSM State	TX Output Level	Duration	Next State Condition
IDLE	High (1)	Indefinite	<code>transmit_trigger = '1'</code>
START	Low (0)	1 bit period	Timer reaches terminal count
DATA	Bit-specific	8 bit periods	<code>bit_counter = 7</code>
STOP	High (1)	1 bit period	Timer reaches terminal count

Synchronous Design: To ensure timing closure and prevent metastability, all state transitions must be synchronized to the master clock. The baud rate **"tick"** signal should be used as a clock enable for the FSM, not as the clock itself. This methodology is highly recommended for keeping the entire design within a single synchronous clock domain.

UART Receiver Architecture

The receiver is slightly more complex as it must synchronize its internal logic to an incoming signal with an unknown phase. This is achieved using 16x oversampling.

- **Oversampling:** The receiver logic operates at 16 times the baud rate, providing high resolution for detecting the precise beginning of the start bit.
- **Start Bit Validation:** While in the `IDLE` state, the receiver monitors the RX line for a high-to-low transition (falling edge). Upon detection, it does not immediately transition. Instead, it waits for 7 or 8 ticks of the 16x clock to reach the midpoint of the start bit.
 - If the line is still low after 7 ticks, the start bit is validated, and the receiver moves to the `DATA` state.
 - If the line returns high, it is dismissed as a noise spike or glitch, and the receiver returns to `IDLE`.
- **Data Sampling:** Once the start bit is validated, the receiver resets its counter and samples all subsequent bits (`D0-D7` and the Stop Bit) exactly every **16 ticks**, ensuring sampling occurs precisely in the middle of each bit's duration.



The receiver FSM includes additional logic for validation and sampling:

- **IDLE:** Resets all counters and continuously monitors the RX line for a high-to-low transition.
- **START:** Waits 7 ticks of the 16x clock for midpoint sampling. Validates the line is still low before moving to `DATA`.
- **DATA:** Collects 8 bits by sampling every 16 ticks. Each bit is shifted into a receiver shift register.
- **STOP:** Waits 16 ticks to sample the stop bit.
 - If the sampled line is High (1), the frame is valid.
 - If the sampled line is Low (0), a **"framing error"** is detected, and the received data is typically discarded before the FSM returns to `IDLE`.

Hardware Pin Mapping

The following table details the physical pin assignments on the Basys 3 board for serial communication:

Note on Pin Naming: The pin labeled `UART_TXD_IN` (B18) is an input to the FPGA (it receives data from the PC), and `UART_RXD_OUT` (A18) is an output from the FPGA (it transmits data to the PC).

Design Signal	Basys 3 Resource	FPGA Pin
clk	100 MHz oscillator	W5
tx_out (FPGA → PC)	USB-UART bridge	A18
rx_in (PC → FPGA)	USB-UART bridge	B18

3 Design Specification

Your task will be to implement the following modules:

- **Baud Generator** – `baud_gen` :
 - Generate `tick_1x` at the baud rate and `tick_16x` at 16× the baud rate.
 - Both ticks must be single-clock pulses (one `clk` cycle wide).
 - Use synchronous active-high reset `rst` .
- **TX Controller (8N1)** – `uart_tx_ctrl` :
 - Output idles high.
 - On a `tx_start` pulse in IDLE, latch `tx_data` and transmit:
 - * start bit = 0
 - * 8 data bits, LSB-first
 - * stop bit = 1
 - Advance one bit per `tick` (clock enable).
 - Provide a `tx_busy` flag asserted while a frame is in progress.
- **RX Controller with 16× Oversampling** – `uart_rx_ctrl` :
 - Include a 2-FF synchronizer for `rx_in` .
 - Detect a valid start bit, wait to the midpoint, then sample each data bit at the midpoint using `tick_16x` .
 - Latch `rx_data` when a valid frame is received.
 - Assert `rx_done` for one cycle on a valid byte.
 - Assert `rx_err` for one cycle on a framing error (bad stop bit).
- **Application Integration** — `uart_top` :

Pressing `send_btn` (btnC) transmits the string `Hello there\r\n` to the PC terminal.

Bytes received from the PC are latched and displayed on `led[7:0]` . `rst` is mapped to btnU (active-high reset).

 - Instantiate `baud_gen` , `btn_debouncer` , `uart_tx_ctrl` , `uart_rx_ctrl` .
 - Use btnU as active-high reset (`rst`).
 - Use btnC as `send_btn` to trigger TX of the message `Hello there\r\n` .
 - Display the most recent received byte on `led[7:0]` .

Required Module Interfaces (Must Match Exactly)

Use the following entity declarations exactly; you will implement the architectures.

Do not rename any entities, ports, generics, or testbenches. The provided simulations depend on exact names and directions.

You will be provided with sample testbenches to test the controllers.

```

1 entity baud_gen is
2   generic (
3     CLK_FREQ   : integer := 100_000_000; -- 100 MHz system clock
4     BAUD_RATE  : integer := 9600         -- Target baud rate
5   );
6   port (
7     clk       : in std_logic;
8     rst       : in std_logic;
9     tick_1x   : out std_logic; -- 1x baud tick (for TX)
10    tick_16x  : out std_logic; -- 16x baud tick (for RX oversampling)
11  );
12 end entity baud_gen;
```

```

1 entity uart_tx_ctrl is
2     port (
3         clk      : in std_logic;           -- System clock (100 MHz)
4         rst      : in std_logic;           -- Synchronous reset
5         tick     : in std_logic;           -- Baud rate tick (1x)
6         tx_start : in std_logic;           -- Start transmission pulse
7         tx_data  : in std_logic_vector(7 downto 0); -- Parallel data to send
8         tx_out   : out std_logic;          -- Serial output line
9         tx_busy  : out std_logic;          -- Transmission in progress
10    );
11 end entity uart_tx_ctrl;

```

```

1 entity uart_rx_ctrl is
2     port (
3         clk      : in std_logic;           -- System clock (100 MHz)
4         rst      : in std_logic;           -- Synchronous reset
5         tick_16x : in std_logic;           -- 16x baud rate tick
6         rx_in    : in std_logic;           -- Serial input line
7         rx_data  : out std_logic_vector(7 downto 0); -- Received parallel data
8         rx_done  : out std_logic;          -- Valid byte received pulse
9         rx_err   : out std_logic;          -- Framing error flag
10    );
11 end entity uart_rx_ctrl;

```

```

1 entity uart_top is
2     generic (
3         CLK_FREQ : integer := 100_000_000; -- System clock frequency (Hz)
4         BAUD_RATE : integer := 9600        -- UART baud rate
5     );
6     port (
7         clk      : in std_logic;           -- 100 MHz system clock
8         rst      : in std_logic;           -- Active-high reset (btnU)
9         send_btn : in std_logic;           -- Send button (btnC)
10        rx_in    : in std_logic;           -- UART RX input from PC
11        tx_out   : out std_logic;          -- UART TX output to PC
12        led      : out std_logic_vector(7 downto 0) -- LED display for received char
13    );
14 end uart_top;

```

Hardware Bring-Up (Basys 3 + Windows + PuTTY)

1. Program the Basys 3 with the generated bitstream (uart_top as synthesis top).
2. Find the COM port in Windows Device Manager (Ports (COM & LPT)).
3. Configure PuTTY:
 - Connection type: Serial
 - Speed: 9600
 - Data bits: 8
 - Parity: None
 - Stop bits: 1
 - Flow control: None

Expected Behaviour

- Press btnC: the terminal prints `Hello there` (then new line due to `\r\n`).
- Type characters: `led[7:0]` updates to the most recently received byte.

4 Deliverables

- VHDL source files implementing the required entities (architectures completed).
- Constraints file `basys3_uart.xdc`.
- Short Report.

5 Troubleshooting

- **Baud mismatch (garbled characters):** verify PuTTY baud matches your design's baud rate.

- **Nothing printed on button press:**
 - Confirm `rst` is not asserted (`btnU`).
 - Confirm `btnC` is wired to `send_btn` and debounced.
- **RX never updates LEDs:**
 - Confirm COM port and wiring (B18 is FPGA RX input).
 - Confirm receiver has 2-FF sync and correct midpoint sampling.
- **Framing errors:**
 - Confirm idle-high, start-low, stop-high.
 - Confirm LSB-first shifting.